Mechanistic Intepretability

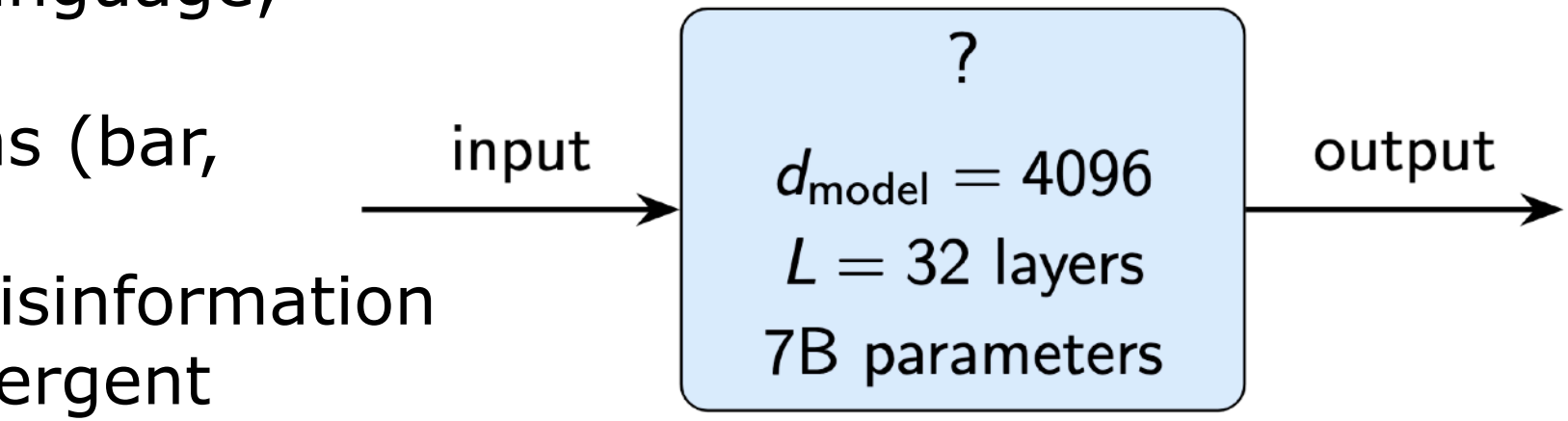
From Understanding to Control

Pasquale Minervini
p.minervini@ed.ac.uk

The Black Box Problem

Modern LLMs can:

- Write code, translate language, reason about math
- Pass professional exams (bar, medical licensing)
- Generate convincing misinformation
- Exhibit unexpected emergent behaviours



But we have almost no idea *how* they do any of this.

Mechanistic Interpretability

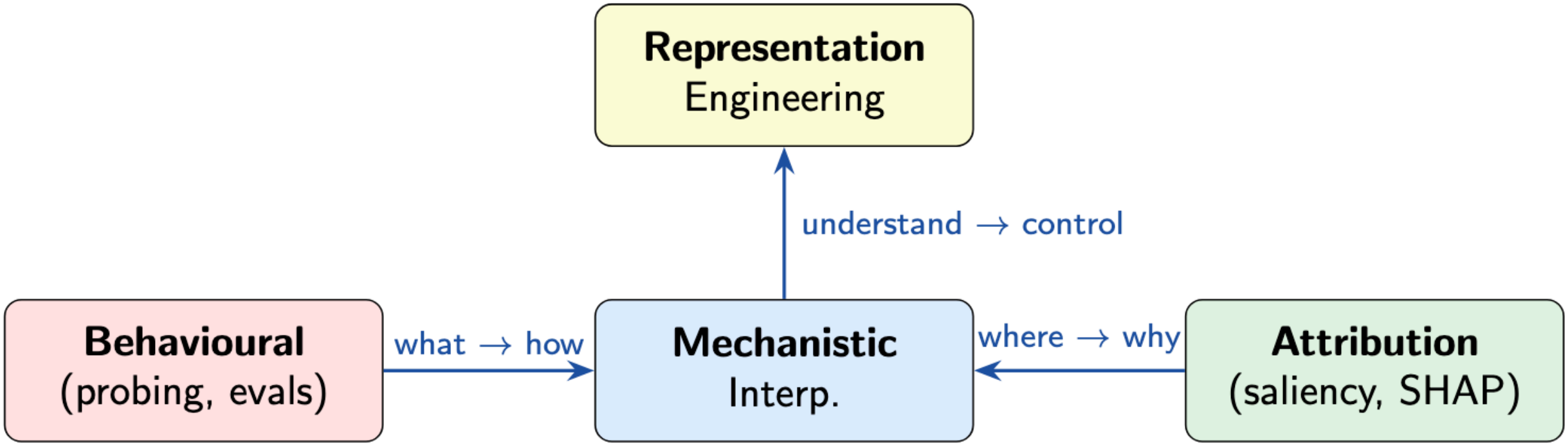
Mechanistic Interpretability (mechinterp) is the attempt to reverse-engineer the *algorithms* implemented by neural networks by identifying the roles of individual components (e.g., neurons, attention heads, layers) and the *circuits* they form.

Key commitments:

- **Mechanistic:** we seek causal explanations, not just correlations
- **Bottom-up:** we look at actual weights and activations
- **Circuits:** we identify subgraphs that implement specific behaviours

Analogy: reverse-engineering a compiled binary → understanding its source code [Olah et al. 2020]

Interpretability Landscape



Behavioural methods ask *what* the model knows; **attribution** asks *where* it loos; **mechanistic** asks *how* it computes; **representation engineering** uses understanding to *control* behaviour.

Why Mechinterp Now?

- **Safety:** as models become more capable, understanding their failure modes becomes *critical*
  - Deceptive alginment, sleeper agents, goal misgeneralisation..
- **Science:** neural networks are the most important computational artefacts of our time — we should def try to understand them
- **Engineering:** better interpretability → better debugging, editing, and control
- **Feasibility:** recent breakthroughs suggest that reverse-engineering is actually possible (e.g., induction heads, IOI circuits, sparse autoencoders at scale)

Key Mechinterp Milestones

Year	Work	Key Contribution
2017	Feature Visualization (Olah et al., 2017)	Visualizing CNN features via optimization
2020	Zoom In (Olah et al., 2020)	Circuits framework for vision models
2021	Math Framework (Elhage et al., 2021)	Residual stream, QK/OV decomposition
2022	Induction Heads (Olsson et al., 2022)	First complete transformer circuit
2022	Superposition (Elhage et al., 2022)	Why neurons are polysemantic
2022	IOI Circuit (Wang et al., 2023)	Circuit for a natural-language task
2022	ROME (Meng et al., 2022)	Causal tracing + knowledge editing
2023	Monosemanticity (Bricken et al., 2023)	Sparse autoencoders for features
2023	RepE / ActAdd (Zou et al., 2023; Turner et al., 2023)	Activation steering via linear directions
2024	Scaling Mono. (Templeton et al., 2024)	SAEs on Claude 3 Sonnet
2024	CAA / ITI (Rimsky et al., 2024; Li et al., 2024)	Robust steering + truthfulness intervention
2024	LoReFT (Wu et al., 2024a)	Interp-inspired parameter-efficient fine-tuning

Transformers, Quick Recap

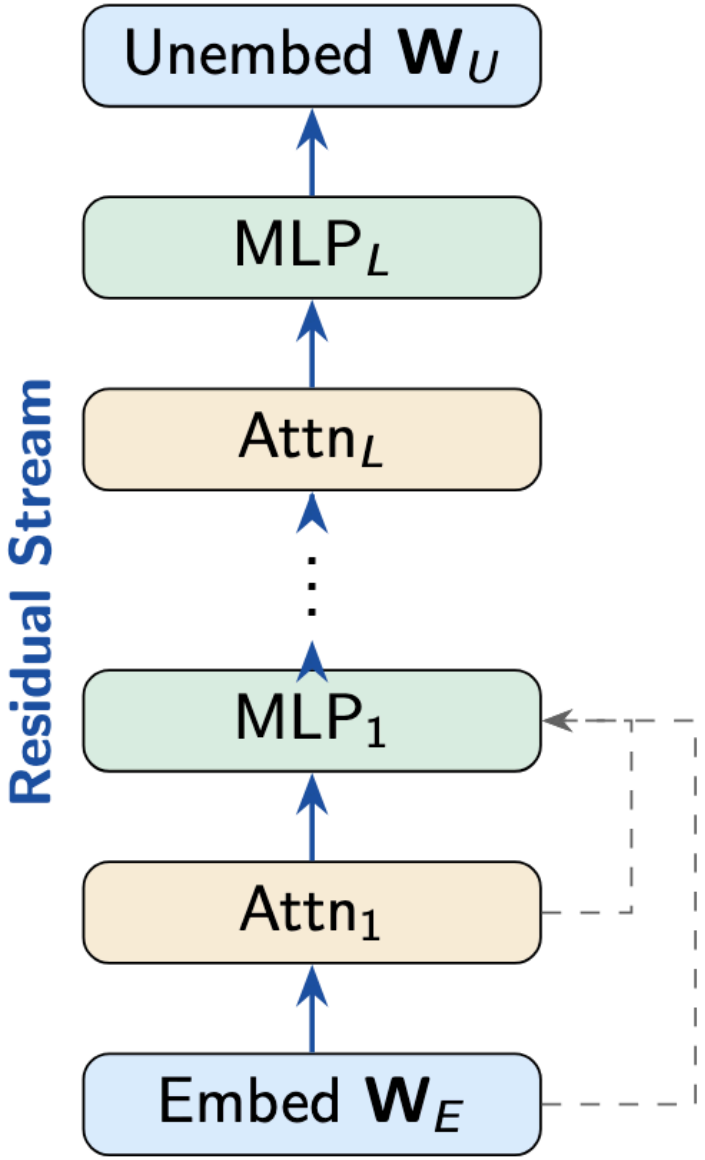
A transformer with L layers processes a sequence of tokens:

$$\mathbf{h}_0 = \mathbf{W}_{\text{emb}}\mathbf{x} + \mathbf{W}_{\text{pos}}$$

$$\mathbf{h}_\ell = \mathbf{h}_{\ell-1} + \text{Att}_\ell\left(\mathbf{h}_{\ell-1}\right)$$

$$+\text{MLP}_\ell\left(\mathbf{h}_{\ell-1} + \text{Att}_\ell\left(\mathbf{h}_{\ell-1}\right)\right)$$

where $\mathbf{h}_\ell \in \mathbb{R}^d$ is the **residual stream** at layer ℓ.



The Residual Stream

The **residual stream** can be seen as a **shared communication channel** through which all components (attention heads, MLPs) read and write.

The output logits can be decomposed as a sum:

$$\text{logits} = \mathbf{W}_{\text{unemb}}\left(\underbrace{\mathbf{W}_{\text{emb}}\mathbf{x}}_{\text{Embedding}} + \underbrace{\sum_{\ell=1}^{L}\text{Att}_{\ell}(\cdot)}_{\text{Head Outputs}} + \underbrace{\sum_{\ell=1}^{L}\text{MLP}_{\ell}(\cdot)}_{\text{MLP Outputs}}\right)$$

- Each component's contribution to the logits can be analysed *independently*
- Components can communicate across layers via the shared stream
- The unembedding matrix $\mathbf{W}_{\text{unemb}}$ converts the stream to logits — we can "read off" any component's contribution

Attention Heads — QK and OV Circuits

Each attention head h at layer ℓ computes the following:

$$\text{head}_{\ell,h}(\mathbf{h}) = \underbrace{\text{softmax}\left(\frac{\left(\mathbf{h}\mathbf{W}_Q^{\ell,h}\right)\left(\mathbf{h}\mathbf{W}_K^{\ell,h}\right)^\top}{\sqrt{d_k}}\right)}_{\text{Attention Pattern (QK circuit)}} \underbrace{\mathbf{h}\mathbf{W}_V^{\ell,h}\mathbf{W}_Q^{\ell,h}}_{\text{OV circuit}}$$

Attention Pattern (QK circuit)
where to move info

OV circuit
what info to move

QK Circuit: determines *which* source positions to attend to:

$$A = \text{softmax}(QK^\top/\sqrt{d_k})$$

OV Circuit: determines *what* information to copy:

$$\text{output} = A \cdot \left(\mathbf{h}\mathbf{W}_V\mathbf{W}_O\right)$$

MLP Layers: Expand, Transform, Project

Each attention head h at layer ℓ computes the following:

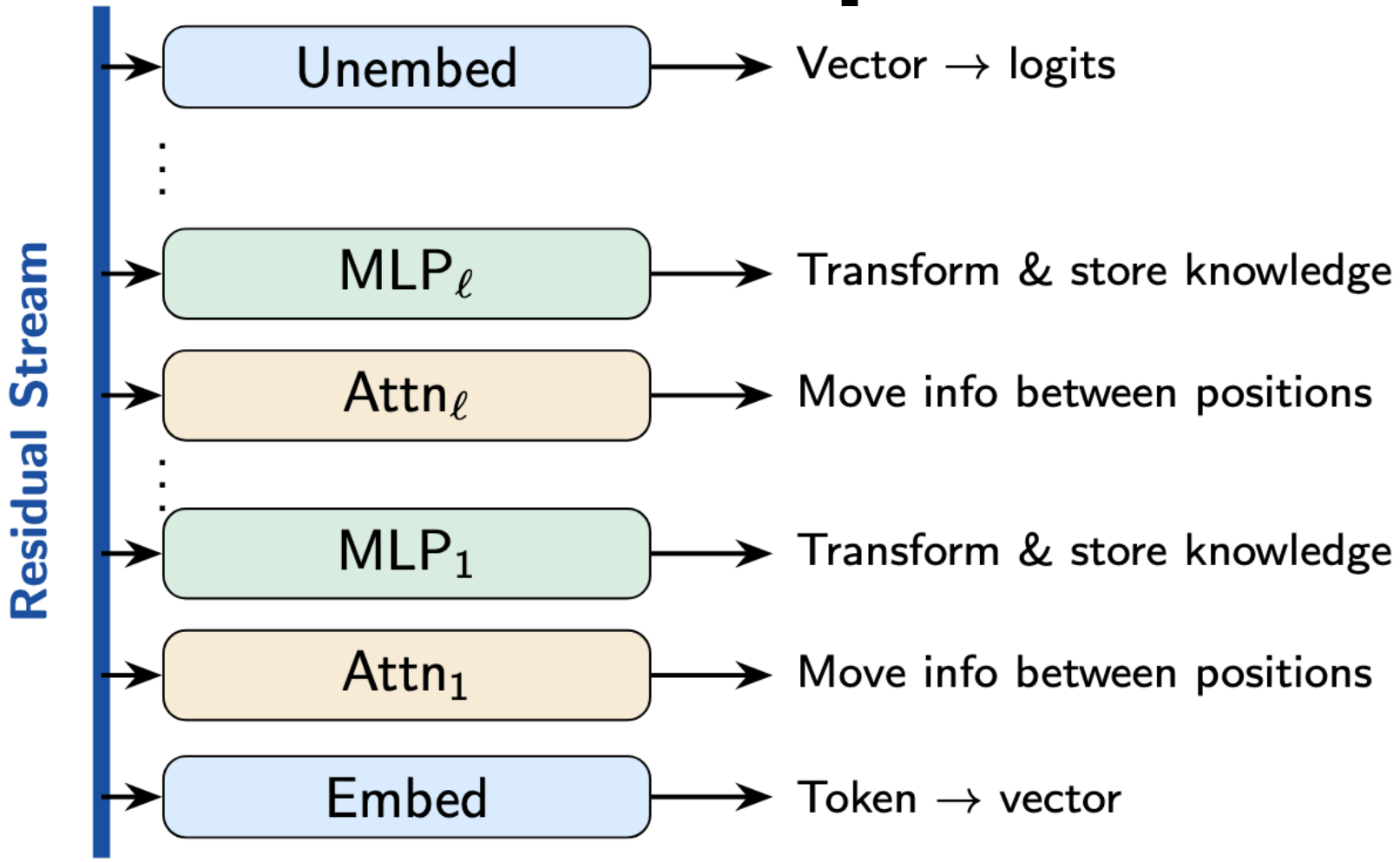
$$\text{MLP}_\ell(\mathbf{h}) = \underbrace{\mathbf{W}_{\text{out}}}_{\text{“Value” matrix}} \cdot \sigma\Big(\underbrace{\mathbf{W}_{\text{in}}}_{\text{“Key” matrix}} \cdot \mathbf{h} \Big)$$

where σ is a non-linearity (ReLU, GELU, SwiGLU..)

MLPs can be seen as Key-Value Memories [Geva et al. 2023]: each row of $\mathbf{W}_{\text{in}}$ act as a **key** (pattern detector); the corresponding column of $\mathbf{W}_{\text{out}}$ is the **value** (what to write to the residual stream when the key fires).

This explains why knowledg editing works — changing a value vector also changes the facts retrieved when key matches [Meng et al. 2022]

The Mechanistic Viewpoint — Summary



Every component **reads from** and **writes to** a shared communication bus — the **residual stream** — which accumulates information across all layers of the model.

Features

A **feature** is a property of the input that a neural network represents internally — a direction in activation space that corresponds to a human-interpretable concept.

Examples of features [Templeton et al. 2024]:

- A direction that activates for text about the Golden Gate Bridge
- A direction that activates when the model is being sycophantic
- A direction encoding Python code with security vulnerabilities
- A direction for requests involving deception

Question: are features encoded by *individual neurons* or by *directions in activation space*?

Superposition

Hope: each neuron represents one concept ("grandmother cell" hyp.)
Reality: while neurons are indeed monosemantic (especially in early/late layers), *most neurons in large models are polysemantic* — they activate for *multiple and seemingly-unrelated* concepts.

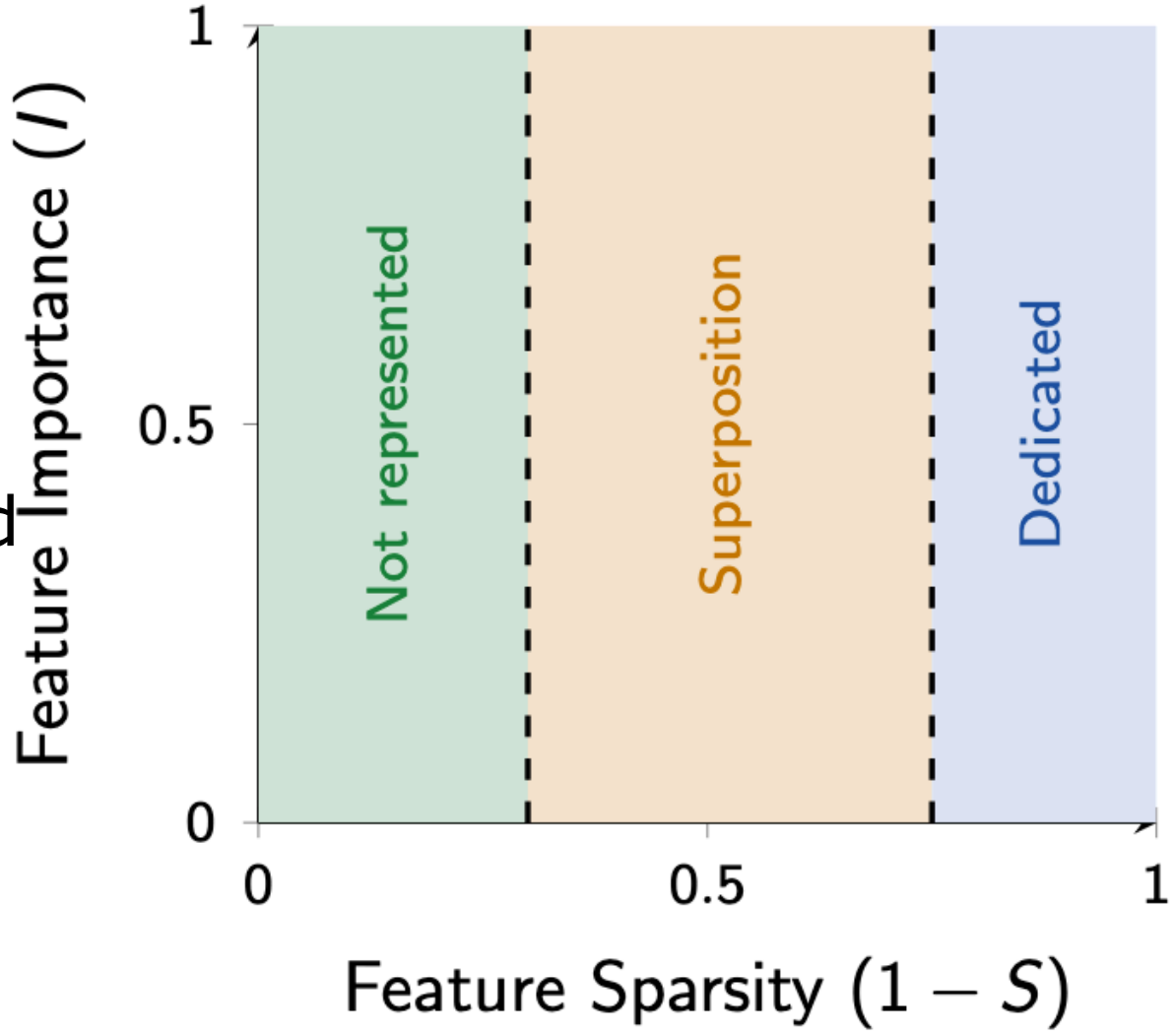
Example: a single polysemantic neuron in GPT-2 can activate for: Academic citations, text about DNA sequences, LaTeX formatting commands, HTTP URLs..

Why? Answer is **superposition** [Elhage et al. 2022]: models represent more features than they have dimensions by encoding features as *almost-orthogonal directions*, with small inference costs.

Superposition Phase Diagram

As feature **sparsity increases** (i.e. features become rarer):

- No representation: feature is too unimportant to dedicate a dimension
- Superposition: feature is encoded in a non-axis-aligned direction, sharing dimensions with other features
- Dedicated dimension: feature is important enough to get its own orthogonal direction



Schematic phase diagram [Elhage et al. 2022]

Linear Representation Hypothesis

Hypothesis [Park et al. 2024]**:** high-level concepts are represented as **linear directions** in the residual stream of LLMs.

Some evidence for this:

- **Truth direction:** Marks and Degmark (2023) fond a linear direction separating "true" from "false" statements in LLM activations
- **Othello board states:** Nanda et al. (2023) showed that board positions are linearly encoded in OthelloGPT
- **Arithmetic:** linear probes can decode intermediate computation results during multi-step arithmetic
- **Causal evidence:** adding/subtracting these direction *changes model behaviour* as predicted

Caveat: not all representations are linear [Li et al. 2023]

Features — Summary

- **Features** are directions in activation space
- **Polysemanticity** happens because models use **superposition** to represent more features than dimensions
- Superposition is an *optimal strategy* when features are **sparse**
- Many (but not all) features are linear directions

Two Challenges:

Observation: how do we *see* what the model represents?
Decomposition: how do we *extract* features from superposition?

Probing — Does the Model Know *X*?

Linear Probe — train a linear classifier on frozen activations to predict a property:

$$\hat{y} = \mathbf{W}_{\text{probe}} \cdot \mathbf{h}_{\ell} + \mathbf{b}_{\text{probe}}$$

Learned

Frozen activation at layer ℓ

If the probes achieves high accuracy, the property is **linearly encoded** in the layer ℓ.

POS tags, syntax trees, NER are linearly encoded [Tenney et al. 2019]
Probing accuracy increases through layers for syntactic properties, and peaks in middle layers for semantic ones
Sentence-level properties can be probed too [Conneau et al. 2018]

Probing — Limitations & Pitfalls

- **The probe may be doing the work:** a sufficiently powerful probe can learn to *compute* the property than just *read* it.
  Control: compare with random representations [Belinkov, 2022]
- **Correlation ≠ Causation:** the property may be encoded but *never used* by the model
  A representation can contain information that downstream computation ignores
- **Layer/Position dependence:** results can vary dramatically across layers, positions, and datasets

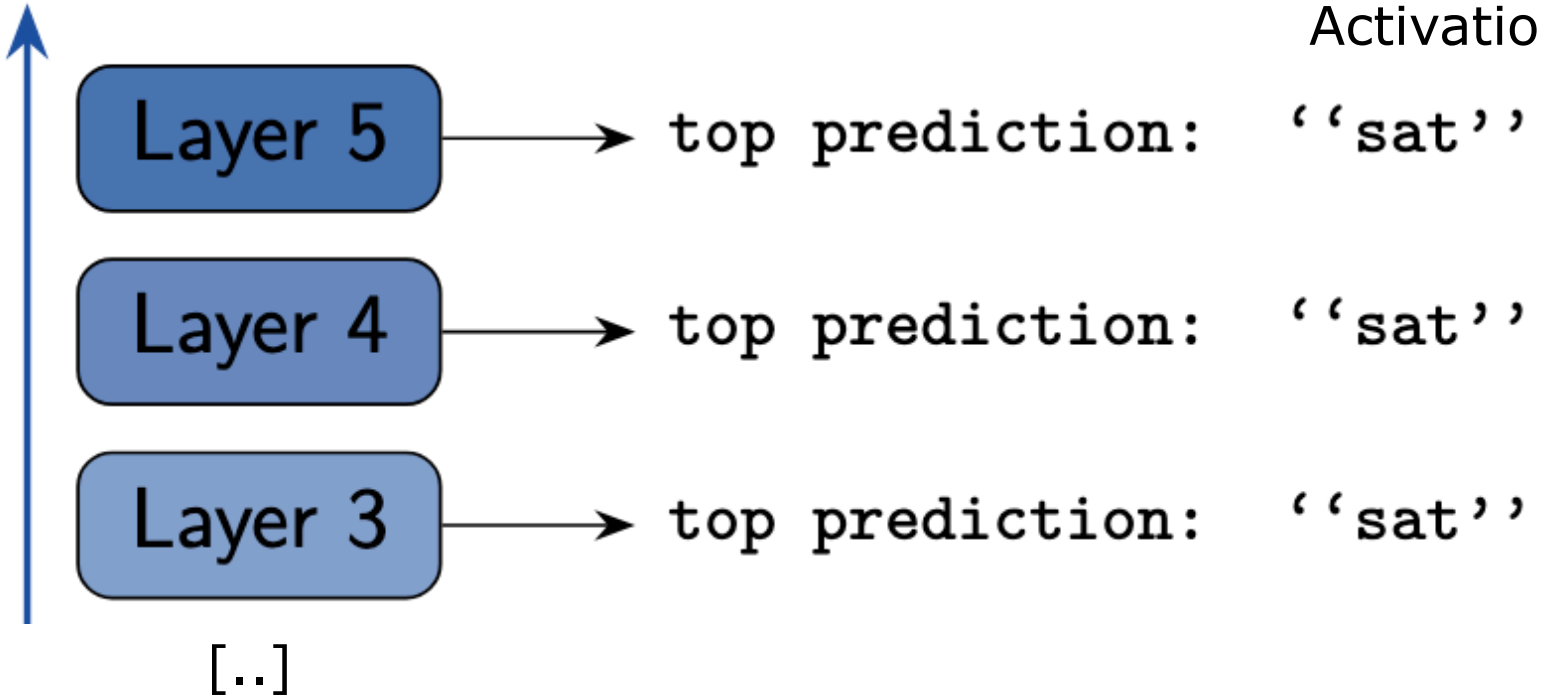
Best Practice: use probing as a *starting point*, then validate with **causal methods** (e.g., activation patching) to confirm that the model actually *uses* the information.

Logit Lens [notalgebraist, 2020]

Idea: apply the unembedding matrix $\mathbf{W}_U$ to intermediate layer activations to "read off" what the model would predict at each layer:

$$\text{logits}_\ell = \mathbf{W}_U \cdot \text{LayerNorm}\left(\mathbf{h}_\ell\right)$$

Activation at layer ℓ



Prediction evolves across layers and often converges in middle layers

Tuned Lens [Belrose et al., 2023]

Problem: logit lens assumes all layers use the same representation space, which is not really true. To solve this, **tuned lens** learns a per-layer affine transformation:

$$\text{logits}_\ell = \mathbf{W}_U \cdot \left(\mathbf{W}_\ell^{\text{lens}} \, \mathbf{h}_\ell + \mathbf{b}_\ell^{\text{lens}} \right)$$

Learned per layer

The lens parameters $\mathbf{W}_\ell^{\text{lens}}$ and $\mathbf{b}_\ell^{\text{lens}}$ are trained to minimise the KL divergence with the final layer predictions. This leads to much more accurate layer-by-layer prediction tracking, especially in early layers.

Used for: tracking when predictions form across layers; detecting anomalous procesing (hallucinations, deception); understanding layer specialisation.

Summary — Observational Methods

Method	What it shows	Causal?
Linear probing	Information encoded at layer ℓ	No
Logit lens	Layer-by-layer predictions	No
Tuned lens	Calibrated layer predictions	No
Attention patterns	Where heads look	No

All of these are observational (correlational)

To be able to make *causal* claims about what components do, we need **interventional methods**.

From Observation to Causation

Fundamental Move:

- **Observation: "**Layer ℓ encodes property P" (e.g., via probing)
- **Causal claim:** "Layer ℓ uses property P to compute the output"

To establish causal claims, we can use **interventions:**

- **Ablation:** remove/zero a component and measure effect on output
- **Activation patching:** replace a component's activation with one from a different input
- **Path patching:** patch along specific computational paths

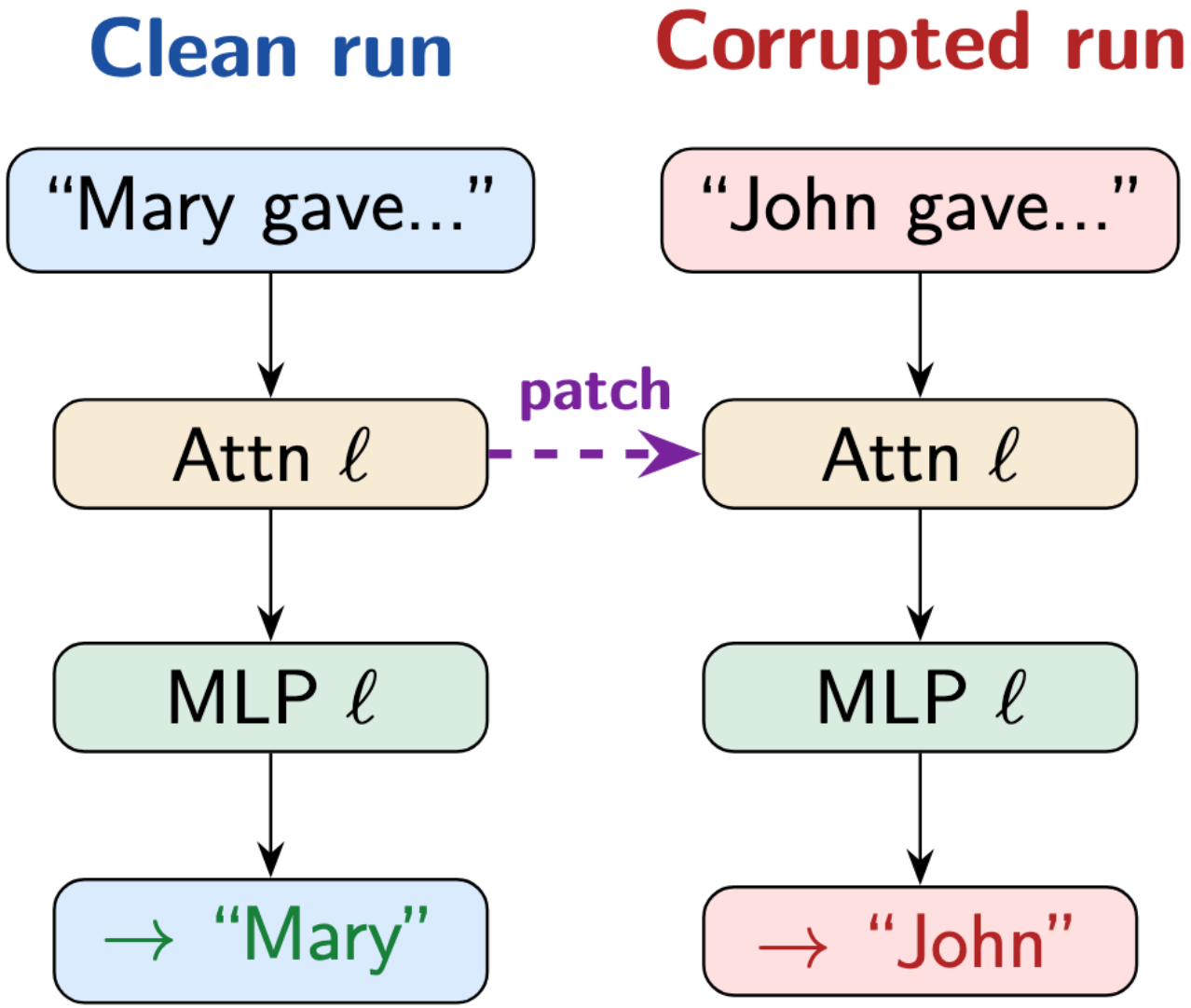
The gold standard from causal inference — intervene on *X*, observe effect on *Y*, while controlling for confounders.

Activation Patching

Steps:

1. Run model on clean input (correct behaviour)
2. Run on corrupted input (wrong behaviour)
3. **Patch**: in the corrupted run, replace activation of component c at layer ℓ with the clean activations
4. **Measure**: does the output recover?

If patching c restores correct beheaviour, c carries the *critical information* for this task.



Causal Tracing — Locating Factual Knowledge

Rank-One Model Editing [ROME; Meng et al. 2022] uses activation patching to locate where factual associations are stored.

Task: "The Eiffel Tower is in _____" → "Paris"

Procedure:

1. **Corrupt** the subject tokens ("Eiffel Tower" → noise)
2. **Restore** activations one layer/position at a time
3. **Measure** recovery of the correct answer

Key Findings:

Early layers at the **last subject token** are decisive

MLP layers (not attention) carry the factual associations

Consistent with MLPs as **key-value memories** — each subject acts as key, and the fact as a value.

Path Tracing — Tracing Computational Paths

Activation patching tells us *which* components matter.
Path patching tells us *how* components interact.

Idea: patch the activation not at component's output, but along a specific *edge* in the computation graph:

Patch at edge $(c_i \to c_j)$: replace c_j's outpu from c_i with clean-run value

Example — in the IOI circuit:

Does head 9.6 use informaton from head 7.3?
Patch only the path from 7.3 → 9.6
If output recovers: 9.6 *depends on* 7.3 for this task

Path patching reveals the **circuit graph**: which components form a computational pipeline [Wang et al. 2023]

Induction Heads

An **Induction Head** [Olsson et al. 2022] is a pair of attention heads that implement the following pattern matching rule:

"… [A][B] … [A]" → predict [B]

Mechanism (2-layer):

1. **Previous-token head** (layer ℓ): attends to the previous position, and writes the *previous token's identity* into the residual stream
2. **Induction head** (layer $\ell' > \ell$): uses QK circuit to find positions where the previous token matches the current token, and uses the OV circuit to copy the next token from that position.

Result: [A][B] … [A] → the induction head attends to the first [B] (since the previous-token head market it as "follows [A]") and copies [B] to predict the next token.

Automated Circuit Discovery

ACDC (Conmy et al., 2023):

- Iteratively prune edges using activation patching
- Remove edges that don't affect output
- Result: sparse circuit subgraph

Attribution Patching (Syed et al., 2023):

- First-order (gradient) approximation to activation patching
- Much faster: 2 forward + 1 backward pass
- Good approximation in practice

Edge Pruning (Bhaskar et al., 2024):

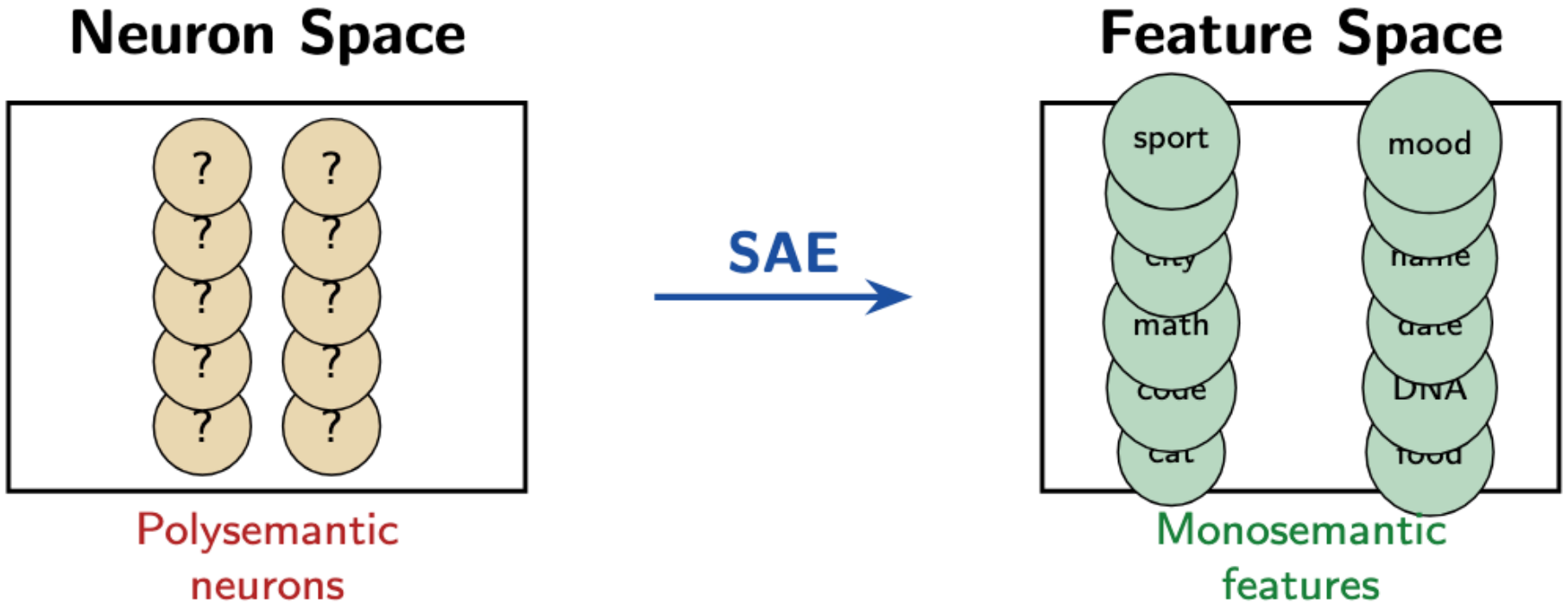
- Frame as optimization: learn binary edge masks
- L_0 regularization for sparsity
- Scales to larger models

Causal Scrubbing (Chan et al., 2022):

- Rigorous test for circuit hypotheses
- Recursively replace activations with "random" inputs consistent with the hypothesis
- Measures if the circuit explanation is sufficient

Superposition Blocks Intepretation

Open Challenge — circuit analysis works at the level of *attention heads and MLP layers* — but what about *individual features* within superposed representations?



Goal — find a transformation from the polysemantic neuron basis to a monosemantic feature basis, even though the feature space is larger.

Dictionary Learning — Idea

Assumption — each activation $\mathbf{h} \in \mathbb{R}^d$ is a sparse combination of **feature directions**:

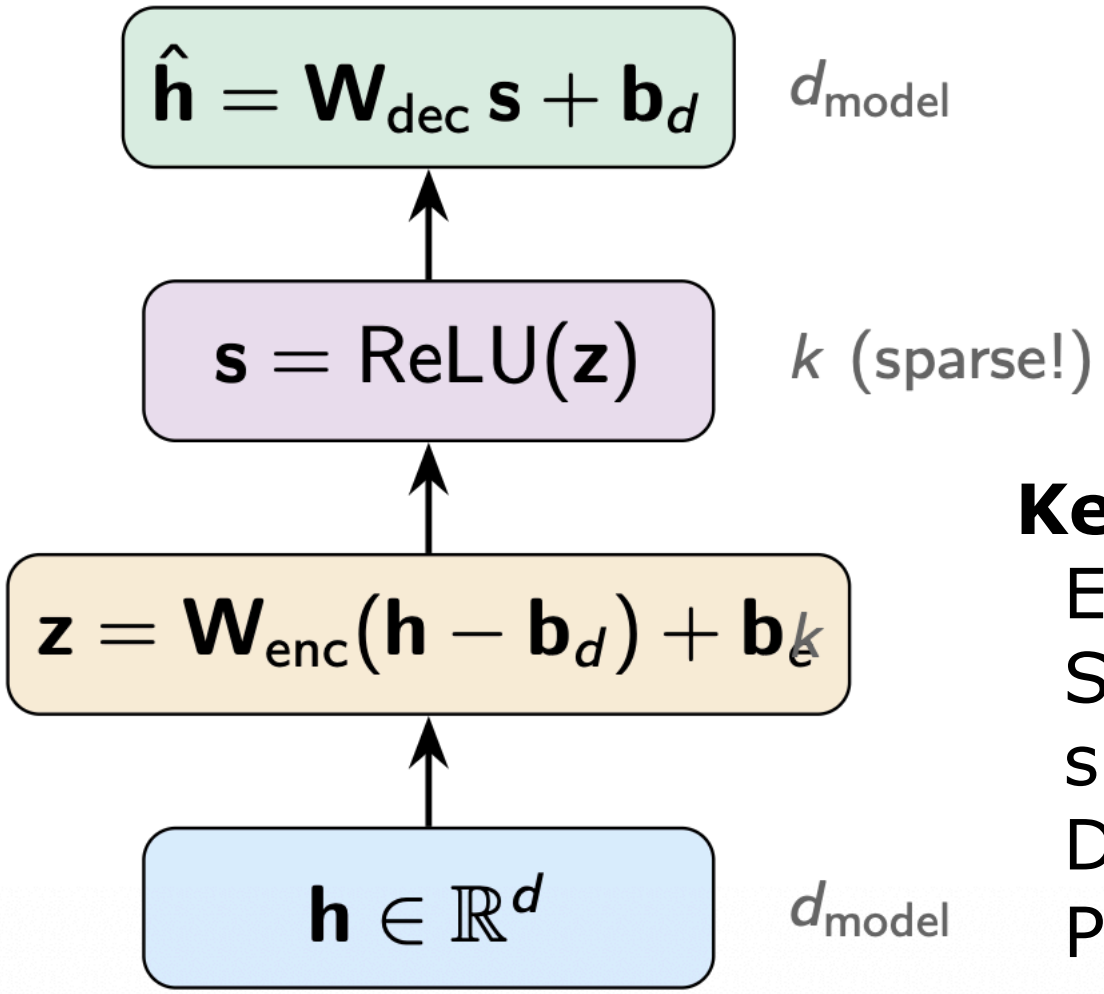
$$\mathbf{h} \approx \underbrace{\mathbf{D}}_{\text{Dictionary}} \cdot \underbrace{\mathbf{s}}_{\substack{\text{Sparse} \\ \text{Codes}}} = \sum_{i}^{k} \underbrace{\mathbf{s}_i}_{\substack{\text{Feature} \\ \text{Activation}}} \cdot \underbrace{\mathbf{d}_i}_{\substack{\text{Feature} \\ \text{Direction}}}$$

with $k \gg d$ and $\|\mathbf{s}\|_0 \ll k$ (i.e. most $\mathbf{s}_i = 0$). The dictionary $\mathbf{D}$ and sparse codes $\mathbf{s}$ are optimised to solve the following optimisation problem:

$$\arg\min_{\mathbf{D},\mathbf{s}} \underbrace{\|\mathbf{h} - \mathbf{D}\mathbf{s}\|_2^2}_{\substack{\text{Reconstruction} \\ \text{Error}}} + \underbrace{\|\mathbf{s}\|_1}_{\substack{\text{Sparsity} \\ \text{Penalty}}}$$

The ℓ_1 penalty encourages activations to be zero, yielding a *sparse decomposition* of each activation vector.

Sparse Autoencoders



Loss function:

$$\mathscr{L} = \|\mathbf{h} - \hat{\mathbf{h}}\|_2^2 + \lambda \sum_i^k |\mathbf{s}_i|$$

Reconstruction Error

Sparsity Penalty

Key design choices:

- Expandion factor k/d (4x-64x)
- Sparsity coefficient λ (reconstruction vs sparsity trade-off)
- Decoder columns $\mathbf{d}_i$ are *unit-normalised*
- Pre-encoder bias $\mathbf{b}_d$

What SAE Features Look Like

Bricken et al. 2023 trained SAEs on a 1-layer transformer and found highly interpretable features:

Monosemantic

Feature for:
- "DNA sequences"
- "Hebrew text"
- "mathematical notation"
- "legal language"

Fine-grained

Feature for:
- "the word *the* after a period"
- "openin brackets"
- "the first word of a line"

Abstract

Feature for:
- "text about deception"
- "sycophantic responses"
- "uncertainty"

Wide variety of SAE features, from concrete/syntactic to abstract/ semantic; most are *much more specific* than individual neurons.

Scaling Monosemanticity

Templeton et al. 2024 scaled SAEs to Claude 3 Sonnet —

- Extracted **millions of features** from middle-level activations
- Features range from concrete to highly abstract:
  - *Golden Gate Bridge* (activates on text/images of GGB)
  - *Code with security vulnerabilities*
  - *Sycophantic praise*
  - *Deceptive behaviour*
- Features are **multilingual and multimodal**: the same feature activates for the concept across languages and modalities

Feature Steering — clamping the Golden Gate Bridge feature to a high value makes Claude mention the bridge in every response — demonstrating causal relevance of SAE features.

To see it in action, you can create a new `Triangle` object and supply the base and height details.

csharp Copy code

```csharp
Triangle goldenGate = new Triangle();
goldenGate.UpdateMeasurements(1.3, 2.2); // base = 1.3 miles, height = 2.2 miles
double areaGoldenGate = goldenGate.GetArea(); // Returns 4.62
```

You can also try this in the interactive shell:

csharp Copy code

```
>>> base_width = 3.0
>>> height = 2.0
>>> area = base_width * height
>>> area
6.0
```

Play around with the code in different ways - maybe add functionality to calculate the area when you know the length of the base and another data Point!